

Online Learning in Statistics and Machine Learning

Lecture 7 – Introduction to Modern Machine Learning

Steven Campbell

Imperial College London Summer School

June 2026

Outline

1. Why online learning?
2. Online gradient descent
3. Bandits and UCB
4. Practical online ML

Why online learning?

Where this lecture fits

Earlier lectures introduced:

- supervised learning, loss functions, cross-validation and regularisation;
- simple classifiers such as k -nearest neighbours;
- decision trees, bagging and random forests.

Today

We focus on **learning over time**: making predictions or decisions as data arrive sequentially, updating the model after each new observation.

This is the setting behind streaming data, adaptive experiments, recommender systems and many modern deployed ML systems.

Batch learning versus online learning

Batch learning

collect data first; train later



d_1 d_2 d_3 d_4 d_5 d_6 d_7 d_8



train model

training set is fixed

Online learning

predict, observe feedback, update



$t=1$ $t=2$ $t=3$ $t=4$ $t=5$ $t=6$

$$W_t \rightarrow W_{t+1}$$

model evolves with the stream

When do we need online learning?

Streaming prediction

- click-through prediction;
- fraud detection;
- sensor monitoring;
- financial forecasting.

Sequential decisions

- which advert to show;
- which treatment arm to try;
- which model to deploy;
- which item to recommend.

Key difference from earlier lectures

The model cannot wait for a clean final training set. It must act now, then learn from what happens.

Prequential evaluation: test before training

Prequential: “Prediction + Sequential”

In online learning, a natural evaluation rule is:

predict first, then update.

Rule

At time t , the learner must predict using only information available. The loss is incurred before seeing the outcome.

This is the online analogue of train/test separation: no look-ahead, no peeking, and no using future outcomes to improve today's prediction.

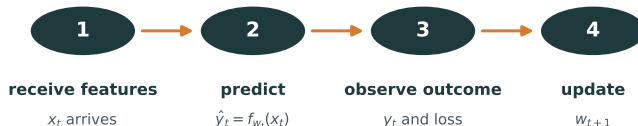
Key idea

Every new data point is first a test point, and only afterwards becomes part of the learner's training history.

The online prediction protocol

At each time $t = 1, 2, \dots, T$:

- 1 receive features x_t ;
- 2 predict $\hat{y}_t = f_{w_t}(x_t)$ using current parameters w_t ;
- 3 observe the true outcome y_t and suffer loss $\ell_t(w_t)$;
- 4 update the model to w_{t+1} .



Evaluate before updating: this mimics performance on genuinely new data.

Three feedback models

Setting	Protocol	Feedback
Batch supervised learning	train once $\rightarrow$ test on held-out data	labels in the training set
Online supervised learning	predict $\rightarrow$ observe $y_t \rightarrow$ update	full feedback : the loss is known
Bandits	choose action $\rightarrow$ observe action's reward $\rightarrow$ update	partial feedback : only the chosen reward is known

This lecture

We will focus on online gradient descent (OGD) and bandits. OGD illustrates a full-feedback setting whereas bandits demonstrate a partial-feedback problem.

Reinforcement learning goes further: actions can also change future states. That is beyond today's scope.

Online gradient descent

From loss functions to online updates

Earlier lectures trained models by minimising an average loss over a fixed training set:

$$\frac{1}{N} \sum_{i=1}^N L(y^{(i)}, f_w(x^{(i)})).$$

In online learning, we see one loss at a time:

$$\ell_t(w) = L(y_t, f_w(x_t)).$$

Online gradient descent idea

After observing ℓ_t , move the parameters a small step in the direction that would have reduced this loss:

$$w_{t+1} = w_t - \eta_t \nabla \ell_t(w_t).$$

This is a **full-feedback** setting: after y_t arrives, we know the loss for the prediction we made.

Online gradient descent: animated view

Each new example defines a loss; OGD updates immediately instead of waiting for a final dataset.

OGD versus SGD: same update, different viewpoint

Online gradient descent

- Data arrive once, in order.
- Evaluate by cumulative loss or regret.
- Use test-then-train predictions.

$$w_{t+1} = w_t - \eta_t \nabla \ell_t(w_t)$$

Stochastic gradient descent

- Data are sampled from a fixed training set.
- Optimise empirical or population risk.
- Evaluate after training on validation/test data.

$$w_{s+1} = w_s - \eta_s \nabla \ell_{i_s}(w_s)$$

Common confusion

The update can look identical. The protocol and evaluation criterion are different.

Example: online linear regression

Suppose

$$f_w(x_t) = w^\top x_t, \quad \ell_t(w) = \frac{1}{2}(y_t - w^\top x_t)^2.$$

Then

$$\nabla \ell_t(w_t) = -(y_t - w_t^\top x_t)x_t = (w_t^\top x_t - y_t)x_t.$$

So the online update is

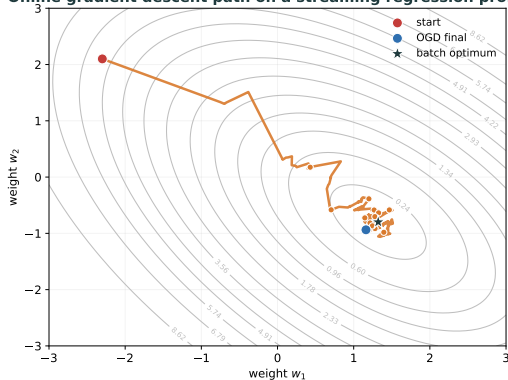
$$w_{t+1} = w_t + \eta_t(y_t - w_t^\top x_t)x_t.$$

Interpretation

If the prediction error $y_t - w_t^\top x_t$ is large, update more; if the error is small, update less.

OGD geometry

Online gradient descent path on a streaming regression problem

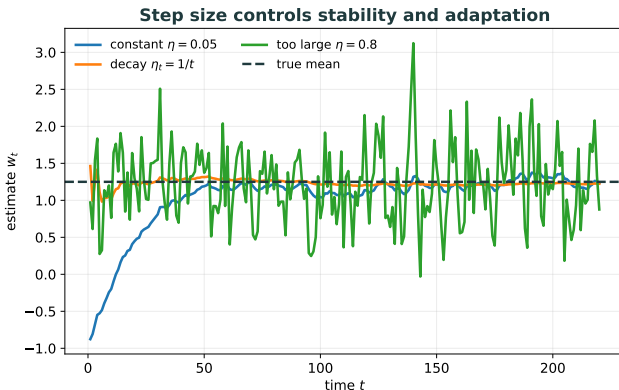


- The orange path shows sequential parameter updates.
- The star is the best fixed parameter vector after seeing all data.
- OGD does not know this optimum in advance.

Question

What happens if the step size is too small? Too large?

The step size matters



- Large step sizes adapt quickly but can be unstable.
- Small step sizes are stable but may adapt slowly.
- Decaying step sizes are useful when the data-generating process is stable.

What should online learning optimise?

A single prediction can be unlucky. We usually assess the learner over time.

Cumulative loss

$$L_T^{\text{online}} = \sum_{t=1}^T \ell_t(w_t).$$

Comparator: best fixed model in hindsight

$$w^* = \arg \min_{w \in \mathcal{W}} \sum_{t=1}^T \ell_t(w).$$

The comparator is allowed to see the whole sequence, but it must use one fixed w for all times.

Regret against the best fixed decision

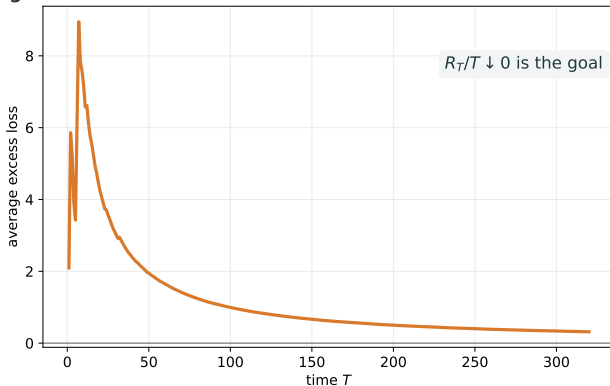
$$R_T = \sum_{t=1}^T \ell_t(w_t) - \min_{w \in \mathcal{W}} \sum_{t=1}^T \ell_t(w).$$

- R_T compares the online learner to the best fixed decision in hindsight.
- We want **sublinear regret**: $R_T = o(T)$.
- Equivalently, average regret vanishes:

$$\frac{R_T}{T} \rightarrow 0.$$

Regret in an online regression example

Regret asks: how far are we from the best fixed model in hindsight?



Interpretation

The learner may make poor early predictions, but the average extra loss can shrink as it adapts.

A useful theorem for OGD

Assume the parameter set $\mathcal{W}$ is convex with diameter at most D , every loss ℓ_t is convex, and gradients are bounded: $\|\nabla \ell_t(w)\|_2 \leq G$.

For projected OGD,

$$w_{t+1} = \Pi_{\mathcal{W}}(w_t - \eta \nabla \ell_t(w_t)), \quad \Pi_{\mathcal{W}}(u) = \text{closest point in } \mathcal{W} \text{ to } u.$$

If the horizon T is known, choosing $\eta = D/(G\sqrt{T})$ gives

$$R_T \leq DG\sqrt{T}.$$

Takeaway

Under idealized convex assumptions, average regret is at most $DG/\sqrt{T}$ and OGD approaches the best fixed decision in hindsight.

In practice, one often uses a schedule such as $\eta_t \propto 1/\sqrt{t}$ instead of assuming T is known.

Mini-exercise: online regression

Suppose the learner predicts a scalar value w_t and observes y_t . Use the squared loss

$$\ell_t(w) = \frac{1}{2}(y_t - w)^2.$$

- ① What is $\nabla \ell_t(w_t)$?
- ② What is the OGD update?
- ③ If $\eta_t = 1/t$, what familiar quantity does w_t become?

Answer sketch

The gradient is $w_t - y_t$ and the update is $w_{t+1} = (1 - \eta_t)w_t + \eta_t y_t$. With $\eta_t = 1/t$ (and any initialization w_1) this tracks the running mean:

$$w_{t+1} = \frac{1}{t} \sum_{s=1}^t y_s, \quad t \geq 1.$$

Bandits and UCB

From prediction to decisions

Online supervised learning is usually **full information**: after predicting, we observe the outcome and can compute the loss.

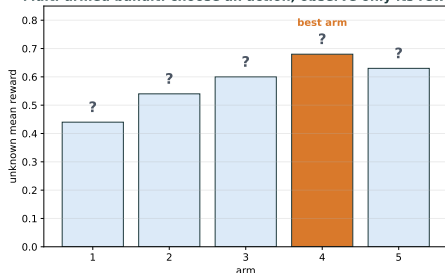
Many real systems have **partial feedback**:

- if we show advert A, we do not observe whether advert B would have worked;
- if we give treatment 1, we do not observe the outcome under treatment 2;
- if we recommend one item, we do not observe responses to all other items.

This leads to the **multi-armed bandit** problem.

Multi-armed bandit setup

Multi-armed bandit: choose an action, observe only its reward



There are K actions, called arms.

- At time t , choose one arm $A_t \in \{1, \dots, K\}$.
- Observe reward $R_t(A_t)$ only for that arm.
- Each arm has unknown mean reward μ_i .

Simple stochastic setting

Assume rewards are bounded, e.g. $R_t(i) \in [0, 1]$, and each arm has a fixed unknown mean μ_i .

The exploration–exploitation dilemma

Exploit

Choose the arm that currently looks best.

- good short-term reward;
- may get stuck with the wrong arm.

Explore

Try uncertain arms to learn more.

- may sacrifice reward now;
- can improve future decisions.

Central question

How should we balance reward now against information for later?

Three simple bandit policies

Random exploration

Choose an arm uniformly at random.
Learns about all arms, but ignores reward information.

Greedy

Pull each arm once, then choose the largest sample mean.
Exploits quickly, but can get stuck after unlucky early rewards.

UCB

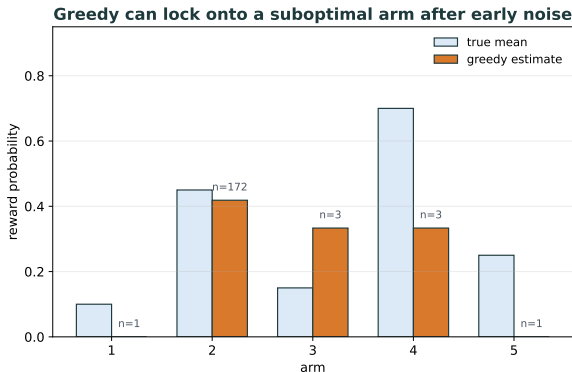
Choose sample mean plus an uncertainty bonus.
Explores selectively until uncertainty shrinks.

Why compare baselines?

Random exploration shows the cost of exploring blindly; greedy shows the risk of exploiting too early.

Why greedy can fail

A greedy strategy chooses the arm with the largest current sample mean.



Problem

Early observations are noisy: if an arm looks bad by chance, greedy may never discover that it was good.

Optimism under uncertainty

UCB stands for **upper confidence bound**.

Principle

Choose the arm that could plausibly be best, not just the arm with the largest current estimate.

For arm i after $t - 1$ rounds, let $\hat{\mu}_i(t - 1)$ be its sample mean reward and $N_i(t - 1)$ the number of pulls. The UCB index is

$$\underbrace{\hat{\mu}_i(t - 1)}_{\text{exploitation}} + \underbrace{\sqrt{\frac{2 \log t}{N_i(t - 1)}}}_{\text{uncertainty bonus}}.$$

Interpretation

The bonus is best read as an *optimism* or *uncertainty* bonus, not as an ordinary 95% confidence interval. It comes from concentration bounds for sample means. Arms with fewer observations receive a larger bonus.

UCB algorithm

- 1 Pull each arm once.
- 2 For $t = K + 1, K + 2, \dots, T$:
 - 1 compute

$$\text{UCB}_i(t) = \hat{\mu}_i(t-1) + \sqrt{\frac{2 \log t}{N_i(t-1)}};$$

- 2 choose

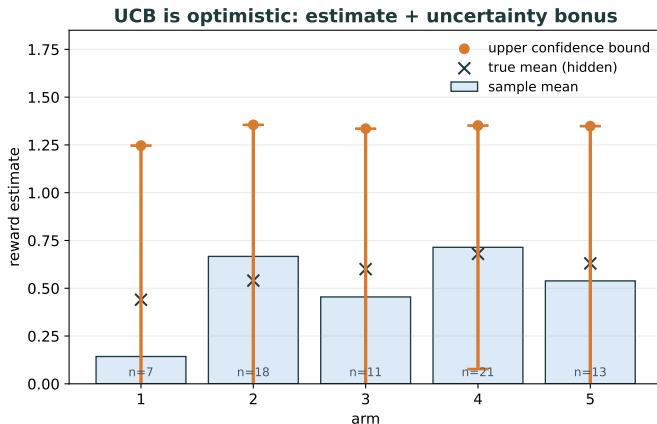
$$A_t = \arg \max_i \text{UCB}_i(t);$$

- 3 observe reward and update $\hat{\mu}_{A_t}$ and N_{A_t} .

Memory needed

For each arm, store only a count and a running average.

UCB uncertainty bonuses



How to read this

The index is the top of the orange bar. Arms with few observations get a larger bonus, so UCB keeps checking uncertain options.

Regret for bandits

Let $\mu_\star = \max_i \mu_i$ be the mean reward of the best arm. The cumulative regret is

$$R_T = T\mu_\star - \mathbb{E} \left[\sum_{t=1}^T R_t(A_t) \right].$$

Equivalently, if $\Delta_i = \mu_\star - \mu_i$, a manipulation gives

$$R_T = \sum_{i=1}^K \Delta_i \mathbb{E}[N_i(T)].$$

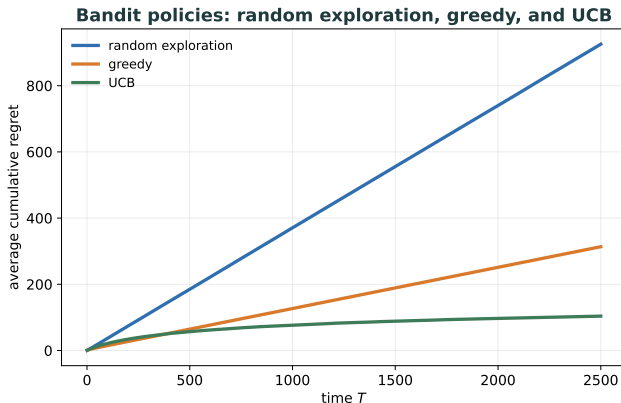
Sign convention

For losses: regret = learner loss – comparator loss. For rewards: regret = comparator reward – learner reward. Same idea, opposite sign.

UCB: animated view

UCB explores arms with large uncertainty bonuses and gradually concentrates on high-reward arms.

Bandit policies in simulation



Takeaway

UCB makes exploration targeted: uncertain arms are explored until their uncertainty shrinks.

What does UCB guarantee?

Under the standard stochastic bandit model with bounded, stationary rewards, UCB has logarithmic regret:

$$R_T = O \left(\sum_{i: \Delta_i > 0} \frac{\log T}{\Delta_i} \right).$$

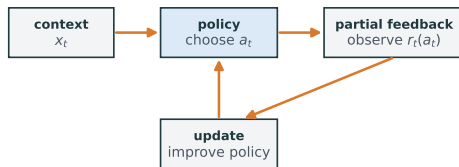
- Bad arms are tried early, then their uncertainty bonuses shrink.
- Clearly suboptimal arms are pulled only a logarithmic number of times.
- Smaller gaps Δ_i are harder: similar arms require more exploration.

Assumption to remember

This guarantee is for a stationary environment. If rewards drift over time, vanilla UCB may become stale.

Contextual bandits: a preview

Similar to bandits, but now actions depend on features!



Contextual bandits combine features with bandit feedback.

- Context x_t : user, patient, document, device state, etc.
- Action a_t : recommendation, treatment, intervention, model choice.
- Feedback: reward only for the action taken.

We will not solve contextual bandits today, but this demonstrates one very natural extension.

Practical online ML

Online classification

For binary classification, a common online pattern is:

- ① receive features x_t ;
- ② compute a score $z_t = w_t^\top x_t$;
- ③ convert the score into a probability

$$p_t = \sigma(z_t) = \frac{1}{1 + e^{-z_t}};$$

- ④ observe $y_t \in \{0, 1\}$;
- ⑤ update using the logistic loss.

The logistic loss is

$$\ell_t(w) = -y_t \log p_t - (1 - y_t) \log(1 - p_t).$$

Connection to earlier lectures

This is the same supervised-learning loss-function idea, but the parameter update is performed after each observation or mini-batch.

Using online methods safely

No future information

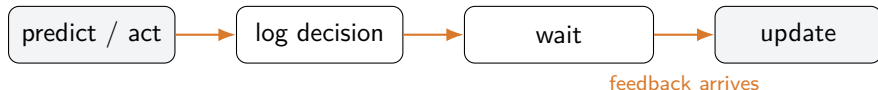
Any preprocessing step must respect time: standardisation, feature selection and hyperparameter tuning cannot use future data.

Safe workflow:

- ① initialise model and preprocessing state using only past data;
- ② for each new batch: transform using the current state;
- ③ predict and record loss before seeing the update;
- ④ after labels arrive, update the model and any allowed state;
- ⑤ monitor performance over time.

Delayed feedback

Sometimes the label or reward does not arrive immediately.

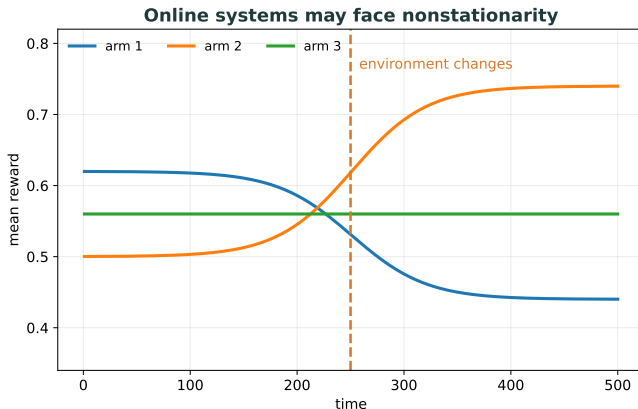


Examples: advert clicks seconds later, purchases days later, medical outcomes weeks later.

Practical consequences

Keep a log of predictions/actions; update only when feedback arrives; distinguish “not observed yet” from “negative outcome” and watch out for delay bias.

Nonstationarity and concept drift



Why this matters

A model that was good yesterday may become stale; online updates must retain enough plasticity to adapt.

Adapting to drift

Common strategies:

- **constant step sizes**: keep adapting instead of fully converging;
- **sliding windows**: train only on recent observations;
- **discounting**: downweight older data gradually;
- **change-point monitoring**: trigger retraining when performance drops;
- **exploration refresh**: in bandits, occasionally re-test old arms.

Tradeoff

Fast adaptation can increase variance. Slow adaptation can miss genuine changes.

Mini-exercise: which setting is it?

For each situation, decide whether it is batch learning, online supervised learning, or a bandit problem.

- ① A spam filter updates every time a user marks an email as spam.
- ② A website chooses one discount offer to show a customer and observes whether the customer buys.
- ③ A fraud model scores transactions as they arrive. Later, each transaction is confirmed as fraud or not fraud.
- ④ A company trains a random forest once per month on a fixed dataset.

Answer sketch

Online supervised; bandit; online supervised; batch learning.

Summary

- Online learning studies sequential prediction and decision-making.
- Online supervised learning has full feedback; bandits have partial feedback.
- The basic protocol is: predict, observe feedback, update.
- Regret compares the learner to the best fixed decision or action in hindsight.
- OGD is a simple full-feedback update; its update resembles SGD but its evaluation is sequential.
- UCB balances exploration and exploitation using “optimism under uncertainty.”
- In practice, online ML requires careful evaluation, preprocessing, delayed-feedback handling and drift monitoring.